Explore | Expand | Enrich

<Codemithra />™

ITERATIVE TOWER OF HANOI

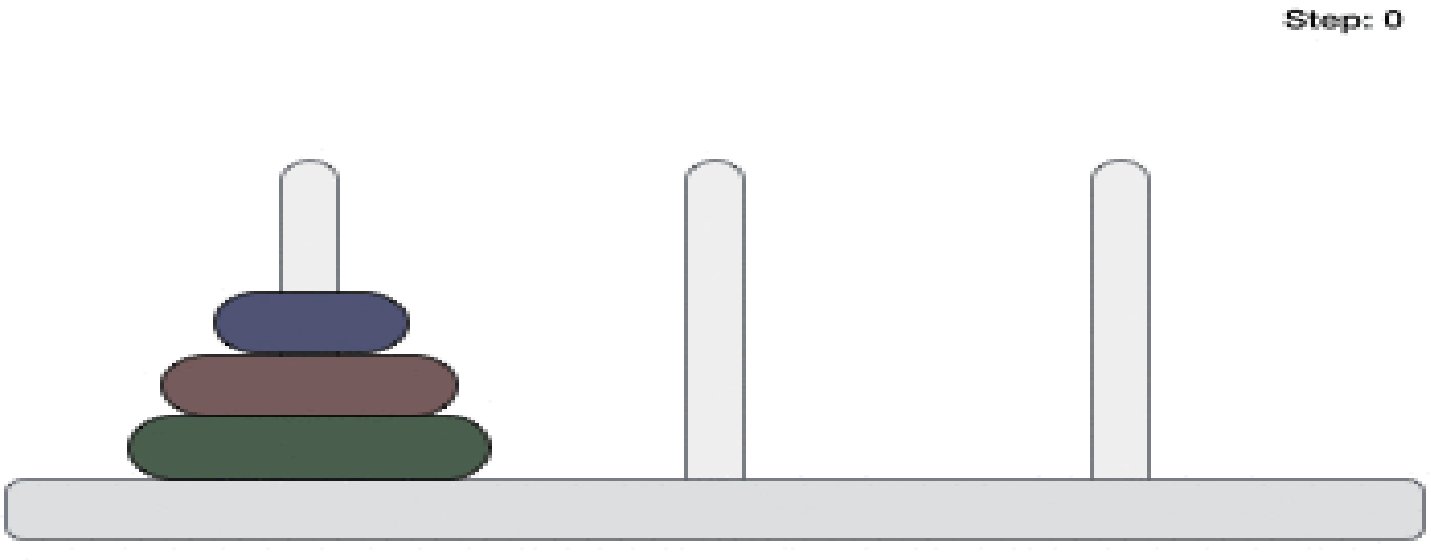
EXPLANATION

- The Tower of Hanoi is a classic mathematical puzzle that involves three rods and a set of disks of different sizes. The objective of the puzzle is to move the entire stack of disks from one rod to another, subject to the following rules:
- Only one disk can be moved at a time.
- A disk can only be placed on top of a larger disk or an empty rod.
- No disk can be placed on top of a smaller disk.

EXPLANATION

The puzzle starts with all the disks stacked in decreasing order of size on one rod (the source rod) and the other two rods (auxiliary and destination rods) are empty. The challenge is to move the entire stack of disks to the destination rod using the auxiliary rod as an intermediate step.

EXPLANATION



ITERATIVE APPROACH

- The iterative approach to solving the Tower of Hanoi problem involves using a stack data structure to keep track of the movements of the disks.
- We maintain three pegs (A, B, and C), and we start with all the disks on peg A.
- The goal is to move all the disks to peg C following the rules of the problem.

ITERATIVE APPROACH ALGORITHM

For an even number of disks:

- Make the legal move between source and auxiliary pegs.
- Make the legal move between source and destination pegs.
- Make the legal move between auxiliary and destination pegs.
- Repeat steps 1-3 until the disks are sorted.

ITERATIVE APPROACH ALGORITHM

For an odd number of disks:

- Make the legal move between source and destination pegs.
- Make the legal move between source and auxiliary pegs.
- Make the legal move between auxiliary and destination pegs.
- Repeat steps 1-3 until the disks are sorted.

ITERATIVE TOWER OF HANOI


```
import java.util.Stack;
public class Main {
    static void towerOfHanoi(int numDisks, char
source, char auxiliary, char destination) {
        // Create three stacks to represent the
three rods
    Stack<Integer> sourceStack = new Stack<>();
Stack<Integer> auxiliaryStack = new Stack<>();
Stack<Integer> destinationStack = new Stack<>();

        // Initialize the source rod with disks
        for (int i = numDisks; i >= 1; i--) {
            sourceStack.push(i);
        }

        // Total number of moves required
        int totalMoves = (int) Math.pow(2,
numDisks) - 1;
```

```
        // Determine the order of pegs for
odd/even number of disks
        char temp;
        if (numDisks % 2 == 0) {
            temp = auxiliary;
            auxiliary = destination;
            destination = temp;
        }

        // Perform iterative Tower of Hanoi
        for (int move = 1; move <= totalMoves;
move++) {
            if (move % 3 == 1) {
                moveDisk(sourceStack,
destinationStack, source, destination);
            } else if (move % 3 == 2) {
                moveDisk(sourceStack,
auxiliaryStack, source, auxiliary);
            }
```

ITERATIVE TOWER OF HANOI

```
else if (move % 3 == 0) {
                moveDisk(auxiliaryStack,
destinationStack, auxiliary, destination);
            }
        }
    }

    static void moveDisk(Stack<Integer>
sourceStack, Stack<Integer> destinationStack, char
source, char destination) {
        if (!sourceStack.isEmpty() &&
(destinationStack.isEmpty() || sourceStack.peek()
< destinationStack.peek())) {

destinationStack.push(sourceStack.pop());
            System.out.println("Move disk " +
destinationStack.peek() + " from " + source + " to
" + destination);
        } else {
```

```
sourceStack.push(destinationStack.pop());
            System.out.println("Move disk " +
sourceStack.peek() + " from " + destination +
" to " + source);
        }
    }

    public static void main(String[] args) {
        int numDisks = 3; // Change the number
of disks as needed
        char source = 'A';
        char auxiliary = 'B';
        char destination = 'C';

        towerOfHanoi(numDisks, source,
auxiliary, destination);
    }
}
```

TIME AND SPACE COMPLEXITY

Time complexity:The time complexity of the iterative Tower of Hanoi algorithm is O(2^n), where n is the number of disks. This means that the number of moves required doubles for each additional disk.

Space Complexity:The space complexity is O(N) because we use three stacks to represent the pegs, and each stack can hold up to n disks.

RECURSIVE ALGORITHM

A recursive algorithm is a problem-solving approach where a function calls itself to solve smaller instances of the same problem. It breaks a complex problem into simpler, identical subproblems, with each recursion reducing the problem size until it reaches a base case. The base case provides the termination condition to stop recursion. Recursive algorithms are commonly used for tasks like tree traversal, sorting, and solving problems with inherent recursive structures.

RECURSIVE ALGORITHM

If there is only one disk, move it from the source rod to the destination rod.

If there are more than one disks:

a. Move the top n-1 disks from the source rod to the auxiliary rod using the destination rod as an auxiliary.

b. Move the nth disk from the source rod to the destination rod.

c. Move the n-1 disks from the auxiliary rod to the destination rod using the source rod as an auxiliary.

ITERATIVE TOWER OF HANOI

```
public class Main {
    public static void main(String[] args) {
        int n = 3;
        towerOfHanoi(n, 'A', 'C', 'B');
    }

public static void towerOfHanoi(int n, char source, char destination, char
auxiliary) {
        if (n == 1) {
         System.out.println("Move disk 1 from " + source + " to " +
destination);
         return;
        }
 towerOfHanoi(n - 1, source, auxiliary, destination);
 System.out.println("Move disk " + n + " from " + source + " to " +
destination);
 towerOfHanoi(n - 1, auxiliary, destination, source);
    }
}
```

ITERATIVE TOWER OF HANOI

```
import java.util.*;
class Main{
    public static void main(String[] args) {
        Scanner s1=new Scanner(System.in);
        System.out.print("Enter the number of disks-");
        int n=s1.nextInt();
        towerofHanoi(n,'A','B','C');
    }
public static void towerofHanoi(int n,char fromrod,char helperrod,char
torod){
    if(n==1)
{
    System.out.println("Move disk from " + fromrod + "->" + torod);
    return;
}
towerofHanoi(n-1,fromrod,torod,helperrod);
towerofHanoi(1,fromrod,helperrod,torod);
towerofHanoi(n-1,helperrod,fromrod,torod);
}}
```

RECURSIVE ALGORITHM TIME AND SPACE COMPLEXITY

Time Complexity: O(2^n) where n is the number of disks due to the doubling of steps with each additional disk.

Space Complexity: O(N) due to the space used by the call stack with a maximum of n simultaneous recursive calls.

INTERVIEW QUESTIONS

1. What is the Tower of Hanoi problem?

Answer: The Tower of Hanoi is a classic mathematical puzzle that involves moving a stack of disks from one rod to another, subject to certain rules.

2. What are the rules of the Tower of Hanoi puzzle?

Answer: The rules are:

Only one disk can be moved at a time.

A larger disk cannot be placed on top of a smaller disk.

Disks can only be moved from the top of one stack to the top of another stack (or an empty rod).

3. How can the Tower of Hanoi problem be solved recursively?

Answer: The problem can be solved by breaking it down into smaller subproblems. For a stack of n disks, you can solve it by:

Moving the top n1 disks to an auxiliary rod.

Moving the largest disk to the destination rod.

Moving the n1 disks from the auxiliary rod to the destination rod.

4. What is the time complexity of the recursive Tower of Hanoi algorithm?

Answer: The time complexity is O(2^n), where n is the number of disks.

INTERVIEW QUESTIONS

5.Are there real-world applications of the Tower of Hanoi problem?

Answer: While it's a classic puzzle, the Tower of Hanoi has applications in computer science, particularly in algorithm analysis and recursion.

https://learn.codemithra.com

Explore | Expand | Enrich

THANK YOU

+91 78150 95095 | codemithra@ethnus.com | www.codemithra.com